МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №3
по дисциплине «Объектно-ориентированное программирование»
Тема: «Создания игрового приложения. Часть третья»

Студент гр. 4388 _____ Кузьмин Г.И.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург
2025

Цель работы.

Доработка базовой архитектуры игрового приложения путем внедрения полноценного игрового цикла, системы игровых уровней с прогрессией сложности и механизмов сохранения/загрузки игрового прогресса в бинарном формате. Изучение и применение принципов обработки исключений, использование идиомы RAII при работе с внешними ресурсами.

Задание.

1. Создать класс(ы) игры, который реализует основной цикл игры, и которому передаются команды от пользователя. Игровой цикл состоит из следующих шагов:
    a. Начало игры
    b. Запуск уровня
    c. Ход игрока. Ход, атака или применение заклинания.
    d. Ход союзников - если имеются
    e. Ход врагов
    f. Ход вражеской базы и башни - если имеются

Условие прохождения уровня студент определяет самостоятельно. Если игрок проигрывает, то игроку должно предлагаться начать заново игру, либо выйти из программы. Все взаимодействие должно происходить через классы игры.

2. Реализовать систему сохранения и загрузки игры. Пользователь должен иметь возможность сохранить игру в любой момент. Пользователь должен иметь возможность загружаться при запуске программы (или выбрать новую), либо во время игры. Сохранения должны оставаться в консистентном состоянии между запусками игры.

3. Добавить обработку исключительных ситуаций для загрузки/сохранения, например невозможность записать в файл, нельзя загрузиться так как файл не существует или в нем некорректные данные.

4. Реализовать переход на следующий уровень, после прохождения уровня. При переходе на следующий уровень создается новое поле другого

размера с более сильными врагами. При переходе на следующий уровень значение жизни игрока восстанавливается, и половина его карточек заклинаний случайным образом удаляется.

Примечание

- Класс игры может знать о игровых сущностях, но не наоборот
- При работе с файлом используйте идиому RAII.
- Исключения должны обязательно обрабатываться, и программа не должна завершаться
- Исключения должны быть информативными (содержать информацию о том, что и где произошло), на разные виды исключительных ситуаций должны быть свои исключения

Архитектура приложения

Архитектура приложения была расширена для поддержки системы уровней и сохранения/загрузки данных. Главный класс игры – Game, который теперь работает как игровой контроллер, управляющий переходами между состояниями главного меню, активной игрой и завершением сессии. Логика усложнения уровней реализована через метод initLevel, который при переходе на каждый следующий этап динамически изменяет параметры окружения: увеличивает игровое поля GameField и пропорционально наращивает характеристики здоровья и урона врагов. При смене уровня восстанавливаются ресурсы игрока с одновременным штрафом: удаления половины накопленных заклинаний.

Для реализации системы сохранения и загрузки был выбран механизм бинарной сериализации данных. Запись состояния осуществляется путем прямого копирования содержимого памяти игровых объектов в файл save.dat с использованием метода write и приведения указателей через reinterpret_cast<char*>. Работа с файловыми потоками std::ofstream и std::ifstream организована в строгом соответствии с идиомой RAII.

Уделено внимание отказоустойчивости системы через обработку исключительных ситуаций. Создана иерархия пользовательских классов

исключений, унаследованных от std::runtime_error, таких как FileException и SaveDataException. Весь блок ввода-вывода и критической логики обернут в конструкции try-catch. Необходимые сообщения выводятся в блок логов игры.

Выполнение работы.

В данной лабораторной работе были реализованы новые классы и переработаны уже имеющиеся.

- **Класс Game** – класс расширен и теперь выполняет роль главного координатора игровых состояний (меню, активная игра, завершение). Он собирает в себе логику прогрессии уровней и взаимодействие с файловой системой.

Поля класса:

1. int currentLevel – хранит номер текущего уровня.
2. bool inMenu – флаг, который показывается необходимость вывода меню.
3. std::vector<std::string> battleLog – список последних игровых событий для вывода на экран

Методы класса:

1. initLevel(int levelNum) – инициализирует параметры нового уровня: увеличивает размер поля, генерирует усиленных врагов, восстанавливает здоровье игрока и применяет штраф к количеству заклинаний.
2. saveGame(const std::string& filename) – выполняет бинарную запись состояния всех объектов (игрока, поля, врагов) в файл.
3. loadGame(const std::string& filename) – считывает бинарные данные, восстанавливает структуру поля и позиции сущностей, используя механизмы обработки исключений.
4. showMenu() – реализует интерфейс главного меню.
5. run() – основной цикл, обеспечивающий переход между меню и игровым процессом.

- **Класс GameException** – базовый класс для всех игровых ошибок. Наследуется от std::runtime_error. Обрабатывает пользовательские исключения, какие могут возникнуть во время работы игры.
- **Класс FileException** – класс, отвечающий за исключения, связанные с файлами. Генерируется при невозможности открытия файла (например, отсутствие save.dat).
- **Класс SaveDataException** – класс сигнализирует о повреждении структуры данных в файле сохранения.
- **Класс Enemy** – класс врага. Был обновлен геттер для получения здоровья. Обновлен конструктор для повышения здоровья и урона.
- **Класс Player** – класс игрока. Были добавлен сеттер здоровья и отредактирован геттер координат.

Далее приведена UML – диаграмма классов для данной лабораторной работы, отражающая связи между классами.

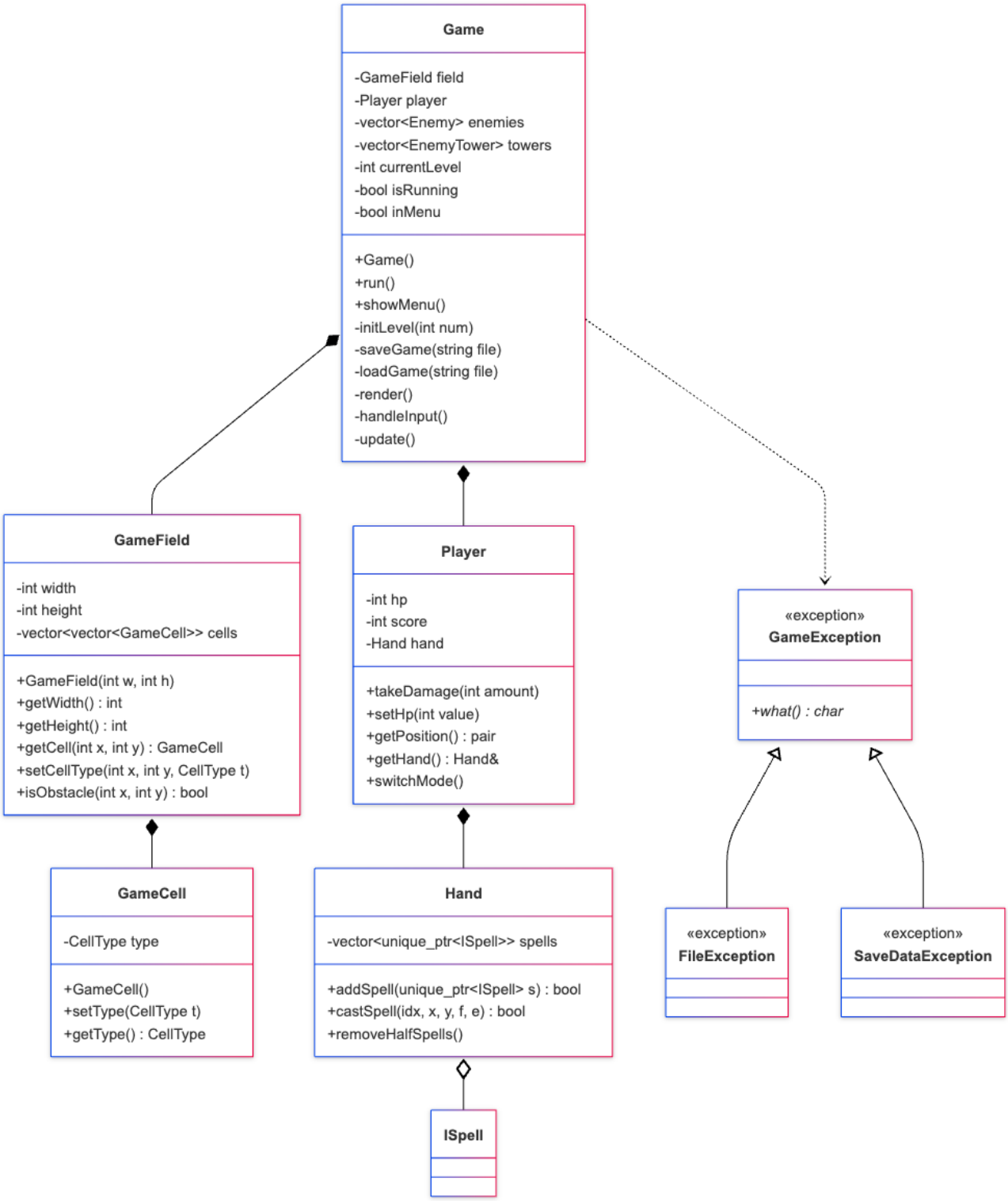

Рис 1. – UML-диаграмма классов

Тестирование.

1. Состояние игры сразу после запуска приложения.

```
=== DUNGEONS AND DRAGONS (LOW COST) ===
1. New Game
2. Load Game
3. Exit
Choice:
```

Рис 2. – Запуск приложения

2. Сохранение игры в файл.

```
================ [ LEVEL 1 ] ================
 [ PLAYER ] HP: 100/100 | Score: 0 | Mode: MELEE
 [ ENEMIES ]
  E1: 40 HP [==========]
  E2: 40 HP [==========]
----------------------------------------
  . . . . . . . . . . . . . . . .
  . P . . . . . . . . # . . . . .
  . . . . . . . . . . . . . # . .
  . . . . . # . . . . . . # . . .
  . . . . # . # . . # . . . # . .
  . . . # . E1. . . . . . . . . .
  . . . . . . . E2. # . . . . . .
  . . . . . . . . . . . . . # . .
  . . . . . # . . . . . . . # . .
  # . . . . . . . . . . . . . . .
  . . . . . . . . . . . . . . . .
  . . . . . . . . . . . . . # . .
  # . . # . # . . . . . # . . . .
  . . . . . . . . . . . . . T . #
  # . . . . . . . . # . . . . . .
  . . . . . . . . . . . . . . # .
---------- [ BATTLE LOG ] ----------
 > Game saved successfully!
------------------------------------
 Spells: [1:Spike Trap]
 [WASD]-Move [F]-Attack [M]-Mode [1-3]-Spell [B]-Buy
 [O]-Save [L]-Load [Q]-Quit
Command:
```

Рис 3. – Сохранение игры

3. Загрузка игры из файла.

```
================ [ LEVEL 1 ] ================
 [ PLAYER ] HP: 100/100 | Score: 0 | Mode: MELEE
 [ ENEMIES ]
  E1: 40 HP [==========]
  E2: 40 HP [==========]
-----------------------------------------
  . . . . . . . . . . . . . . . .
  . P . . . . . . . . # . . . . .
  . . . . . . . . . . . . . # . .
  . . . . . # . . . . . . # . . .
  . . . . # . # . . # . . . # . .
  . . . # . E1. . . . . . . . . .
  . . . . . . . E2. # . . . . . .
  . . . . . . . . . . . . # . . .
  . . . . . # . . . . . . . # . .
  # . . . . . . . . . . . . . . .
  . . . . . . . . . . . . . . . .
  . . . . . . . . . . . . . # . .
  # . . # . # . . . . . # . . . .
  . . . . . . . . . . . . . T . #
  # . . . . . . . . # . . . . . .
  . . . . . . . . . . . . . . # .
---------- [ BATTLE LOG ] ----------
 > Game loaded from file!
-----------------------------------
 Spells:
 [WASD]-Move [F]-Attack [M]-Mode [1-3]-Spell [B]-Buy
 [O]-Save [L]-Load [Q]-Quit
Command:
```

Рис4. – Загрузка игры из сохранения

4. Попытка загрузки без файла.

```
1. New Game
2. Load Game
3. Exit
Choice: 2
Error: Could not open file save.dat Press Enter...
```

Рис. 5 – Загрузка игры без файла

5. Переход на следующий уровень.

```
================ [ LEVEL 2 ] ================
 [ PLAYER ] HP: 100/100 | Score: 200 | Mode: RANGED
 [ ENEMIES ]
  E1: 50 HP [==========]
  E2: 50 HP [==========]
  E3: 50 HP [==========]
-----------------------------------------
  . . . . . . . . . . . . . . . . . . .
  . P . . . . . . . . . . . . . . . . .
  . . . . . . . . . . . . . # . . # # .
  . . . . . # . . # . . . . . . . . . .
  . . . . . # . . . . . . . . . . . . .
  # . . . . E1. . . . . . # # . . . .
  . . . . . . . E2. . . # . . . . . .
  . . . . . . . . . E3# # . . . # #
  . . # . . . . . . . . . . . . . # .
  . . . . # . # . . . . . . . . . . .
  . . . . # . . . . . # . . . . . . #
  . . . . . . # . . # . . . . # . . .
  # # . . . . . . . # . . # . . . . .
  . . . . . # . . . . . . . . . # . .
  . . . . . . . . . . # . . . T # .
  . . . . . . . . # . . . . # . . .
  . . . . . . . . . . . . # . . . . .
---------- [ BATTLE LOG ] ----------
 > Hit E1!
 > Hit E1!
 > Hit E1!
 > Enemy defeated! +100 Score
 > Level UP! HP restored, half spells lost.
-----------------------------------
 Spells: [1:Spike Trap]
 [WASD]-Move [F]-Attack [M]-Mode [1-3]-Spell [B]-Buy
 [O]-Save [L]-Load [Q]-Quit
Command:
```

Рис. 6 – Переход на следующий уровень

Вывод

В ходе выполнения лабораторной работы были разработаны классы для сохранения и загрузки процесса игры, обработки ошибок, возникающих во время выполнения программы. Был разработан игровой цикл. При реализации были соблюдены принципы ООП и требования по оформлению кода. Тестирование не выявило критических ошибок. Функции работают штатно.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: Makefile

```
CXX = g++
CXXFLAGS = -Wall -Wextra -std=c++17

TARGET = game

OBJS = main.o Game.o GameField.o Player.o Enemy.o GameCell.o Hand.o
\
            DirectDamageSpell.o       AreaDamageSpell.o       TrapSpell.o
EnemyTower.o

all: $(TARGET)

$(TARGET): $(OBJS)
     $(CXX) $(CXXFLAGS) -o $(TARGET) $(OBJS)

main.o: main.cpp Game.h
     $(CXX) $(CXXFLAGS) -c main.cpp

Game.o: Game.cpp Game.h GameField.h Player.h Enemy.h GameExceptions.h
DirectDamageSpell.h AreaDamageSpell.h TrapSpell.h
     $(CXX) $(CXXFLAGS) -c Game.cpp

GameField.o: GameField.cpp GameField.h GameCell.h Enums.h
     $(CXX) $(CXXFLAGS) -c GameField.cpp

Player.o: Player.cpp Player.h Enums.h Hand.h
     $(CXX) $(CXXFLAGS) -c Player.cpp

Enemy.o: Enemy.cpp Enemy.h Player.h GameField.h
     $(CXX) $(CXXFLAGS) -c Enemy.cpp

GameCell.o: GameCell.cpp GameCell.h Enums.h
     $(CXX) $(CXXFLAGS) -c GameCell.cpp

Hand.o: Hand.cpp Hand.h ISpell.h
     $(CXX) $(CXXFLAGS) -c Hand.cpp

DirectDamageSpell.o:    DirectDamageSpell.cpp    DirectDamageSpell.h
ISpell.h Enemy.h GameField.h
     $(CXX) $(CXXFLAGS) -c DirectDamageSpell.cpp

AreaDamageSpell.o:  AreaDamageSpell.cpp  AreaDamageSpell.h  ISpell.h
Enemy.h GameField.h
     $(CXX) $(CXXFLAGS) -c AreaDamageSpell.cpp

TrapSpell.o: TrapSpell.cpp TrapSpell.h ISpell.h GameField.h Enums.h
     $(CXX) $(CXXFLAGS) -c TrapSpell.cpp

EnemyTower.o: EnemyTower.cpp EnemyTower.h Player.h
     $(CXX) $(CXXFLAGS) -c EnemyTower.cpp

clean:
```

```
	rm -f $(OBJS) $(TARGET)
```

Название файла: Game.h

```
#ifndef GAME_H
#define GAME_H

#include <vector>
#include <string>
#include "GameField.h"
#include "Player.h"
#include "Enemy.h"
#include "EnemyTower.h"
#include <iostream>

class Game {
private:
    GameField field;
    Player player;
    std::vector<Enemy> enemies;
    std::vector<std::string> battleLog;
    std::vector<EnemyTower> towers;

    int currentLevel;
    bool isRunning;
    bool inMenu;

    void addLog(const std::string& msg);
    void render();
    void handleInput();
    void update();
    void initLevel(int levelNum);
    void saveGame(const std::string& filename);
    void loadGame(const std::string& filename);
    void showMenu();

public:
    Game();
    void run();
};

#endif
```

Название файла: Game.cpp

```
#include "Game.h"
#include "Player.h"
#include "Enemy.h"
#include "GameField.h"
#include "GameCell.h"
#include "Enums.h"
#include "DirectDamageSpell.h"
#include "AreaDamageSpell.h"
#include "TrapSpell.h"
#include "GameExceptions.h"
#include <iostream>
#include <fstream>
#include <cstdlib>
#include <ctime>
```

```
#include <cmath>

Game::Game() : field(15, 15), player(100), currentLevel(1),
isRunning(true), inMenu(true) {
    // initLevel(currentLevel);
}

void Game::initLevel(int levelNum) {
    // Новое поле
    int size = 15 + levelNum;
    if (size > 25) size = 25;
    field = GameField(size, size);

    // Игрок
    player.setPosition(1, 1);
    player.setHp(100);

    if (levelNum == 1) {
        if (player.getHand().getSpells().empty()) {
            int r = std::rand() % 3;
            if (r == 0)
player.getHand().addSpell(std::make_unique<DirectDamageSpell>());
            else if (r == 1)
player.getHand().addSpell(std::make_unique<AreaDamageSpell>());
            else
player.getHand().addSpell(std::make_unique<TrapSpell>());
        }
    } else {
        player.getHand().removeHalfSpells();
        addLog("Level UP! HP restored, half spells lost.");
    }

    // Враги
    enemies.clear();
    for (int i = 0; i < 1 + levelNum; ++i) {
        enemies.push_back(Enemy(30 + levelNum * 10, 5 + levelNum *
2, 5 + i * 2, 5 + i));
    }

    // Башни
    towers.clear();
    towers.push_back(EnemyTower(size - 3, size - 3, 10 + levelNum,
3, 2));
}

void Game::saveGame(const std::string& filename) {
    std::ofstream ofs(filename, std::ios::binary);
    if (!ofs.is_open()) throw FileException(filename);

    // Уровень, HP, Score
    ofs.write(reinterpret_cast<char*>(&currentLevel), sizeof(int));
    int hp = player.getHp();
    ofs.write(reinterpret_cast<char*>(&hp), sizeof(int));
    int score = player.getScore();
    ofs.write(reinterpret_cast<char*>(&score), sizeof(int));

    // Координаты игрока
```

```
        auto pos = player.getPosition();
        int px = pos.first;
        int py = pos.second;
        ofs.write(reinterpret_cast<char*>(&px), sizeof(int));
        ofs.write(reinterpret_cast<char*>(&py), sizeof(int));

        // Параметры поля
        int w = field.getWidth();
        int h = field.getHeight();
        ofs.write(reinterpret_cast<char*>(&w), sizeof(int));
        ofs.write(reinterpret_cast<char*>(&h), sizeof(int));

        // Клетки поля
        for (int y = 0; y < h; ++y) {
            for (int x = 0; x < w; ++x) {
                CellType type = field.getCell(x, y).getType();
                ofs.write(reinterpret_cast<char*>(&type),
sizeof(CellType));
            }
        }

        // Враги
        size_t eCount = enemies.size();
        ofs.write(reinterpret_cast<char*>(&eCount), sizeof(size_t));
        for (const auto& e : enemies) {
            int ex = e.getX();
            int ey = e.getY();
            int ehp = e.getHp();
            ofs.write(reinterpret_cast<char*>(&ex), sizeof(int));
            ofs.write(reinterpret_cast<char*>(&ey), sizeof(int));
            ofs.write(reinterpret_cast<char*>(&ehp), sizeof(int));
        }

        addLog("Game saved successfully!");
    }

    void Game::loadGame(const std::string& filename) {
        std::ifstream ifs(filename, std::ios::binary);
        if (!ifs.is_open()) throw FileException(filename);

        // Читаем базовые параметры
        int level, hp, score, px, py;
        ifs.read(reinterpret_cast<char*>(&level), sizeof(int));
        ifs.read(reinterpret_cast<char*>(&hp), sizeof(int));
        ifs.read(reinterpret_cast<char*>(&score), sizeof(int));
        ifs.read(reinterpret_cast<char*>(&px), sizeof(int));
        ifs.read(reinterpret_cast<char*>(&py), sizeof(int));

        currentLevel = level;

        // Читаем и восстанавливаем поле
        int w, h;
        ifs.read(reinterpret_cast<char*>(&w), sizeof(int));
        ifs.read(reinterpret_cast<char*>(&h), sizeof(int));

        field = GameField(w, h);
        for (int y = 0; y < h; ++y) {
```

```
            for (int x = 0; x < w; ++x) {
                CellType type;
                ifs.read(reinterpret_cast<char*>(&type),
sizeof(CellType));
                field.setCellType(x, y, type);
            }
        }

        // Применяем данные игрока
        player.setHp(hp);
        player.addScore(score - player.getScore());
        player.setPosition(px, py);

        // Загрузка врагов
        size_t eCount;
        ifs.read(reinterpret_cast<char*>(&eCount), sizeof(size_t));
        enemies.clear();
        for (size_t i = 0; i < eCount; ++i) {
            int ex, ey, ehp;
            ifs.read(reinterpret_cast<char*>(&ex), sizeof(int));
            ifs.read(reinterpret_cast<char*>(&ey), sizeof(int));
            ifs.read(reinterpret_cast<char*>(&ehp), sizeof(int));
            enemies.push_back(Enemy(ehp, 5 + currentLevel * 2, ex, ey));
        }

        // Восстанавливаем башни
        towers.clear();
        towers.push_back(EnemyTower(w - 3, h - 3, 10 + currentLevel, 3,
2));

        addLog("Game loaded from file!");
    }

    void Game::showMenu() {
        system("clear");
        std::cout << "=== DUNGEONS AND DRAGONS (LOW COST) ==="  <<
std::endl;
        std::cout << "1. New Game" << std::endl;
        std::cout << "2. Load Game" << std::endl;
        std::cout << "3. Exit" << std::endl;
        std::cout << "Choice: ";

        char choice;
        std::cin >> choice;
        if (choice == '1') {
            currentLevel = 1;
            initLevel(currentLevel);
            inMenu = false;
        } else if (choice == '2') {
            try {
                loadGame("save.dat");
                inMenu = false;
            } catch (const GameException& e) {
                std::cout << e.what() << " Press Enter..." << std::endl;
                std::cin.ignore(); std::cin.get();
            }
        } else {
```

```
            isRunning = false;
        }
    }

    void Game::render() {
        system("clear");

        // Шапка
        std::cout << "================ [ LEVEL " << currentLevel << " ]
================" << std::endl;
        std::cout << " [ PLAYER ] HP: " << player.getHp() << "/100 |
Score: " << player.getScore()
                  << " | Mode: " << (player.getMode() ==
CombatMode::MELEE ? "MELEE" : "RANGED") << std::endl;

        // Список врагов
        std::cout << " [ ENEMIES ] " << std::endl;
        if (enemies.empty()) {
            std::cout << "  Area clear! Go to the next level." <<
std::endl;
        } else {
            for (size_t i = 0; i < enemies.size(); ++i) {
                int bars = (enemies[i].getHp() * 10) /
enemies[i].getMaxHp();
                std::cout << "  E" << i+1 << ": " << enemies[i].getHp()
<< " HP [";
                for(int b=0; b<10; b++) std::cout << (b < bars ? "=" : "
");
                std::cout << "]" << std::endl;
            }
        }
        std::cout << "-------------------------------------------" <<
std::endl;

        // Отрисовка карты
        for (int y = 0; y < field.getHeight(); ++y) {
            std::cout << "  ";
            for (int x = 0; x < field.getWidth(); ++x) {
                // Приоритет: Игрок > Враг > Башня > Клетка поля
                if (x == player.getPosition().first && y ==
player.getPosition().second) {
                    std::cout << "P ";
                } else {
                    int enemyIdx = -1;
                    for (size_t i = 0; i < enemies.size(); ++i) {
                        if (enemies[i].getX() == x && enemies[i].getY()
== y) {
                            enemyIdx = i; break;
                        }
                    }

                    if (enemyIdx != -1) {
                        std::cout << "E" << enemyIdx + 1;
                    } else {
                        bool towerFound = false;
                        for (const auto& t : towers) {
```

```
                            if (t.getX() == x && t.getY() == y) {
                                std::cout << "T "; towerFound = true;
break;
                            }
                        }

                        if (!towerFound) {
                            CellType t = field.getCell(x, y).getType();
                            if (t == CellType::TRAP) std::cout << "^ ";
                            else if (t == CellType::OBSTACLE) std::cout
<< "# ";
                            else std::cout << ". ";
                        }
                    }
                }
            }
            std::cout << std::endl;
        }

        // Лог событий
        std::cout << "---------- [ BATTLE LOG ] ----------" << std::endl;
        for (const auto& log : battleLog) {
            std::cout << " > " << log << std::endl;
        }
        std::cout << "-----------------------------------" << std::endl;

        // Управление
        auto& spells = player.getHand().getSpells();
        std::cout << " Spells: ";
        for (size_t i = 0; i < spells.size(); ++i) std::cout << "[" <<
i+1 << ":" << spells[i]->getName() << "] ";
        std::cout << "\n [WASD]-Move [F]-Attack [M]-Mode [1-3]-Spell [B]-
Buy" << std::endl;
        std::cout << " [O]-Save [L]-Load [Q]-Quit" << std::endl;
    }

    void Game::handleInput() {
        char input;
        std::cout << "Command: ";
        std::cin >> input;
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');

        bool turnTaken = false;

        try {
            if (input == 'q') { isRunning = false; return; }
            if (input == 'o') { saveGame("save.dat"); return; }
            if (input == 'l') { loadGame("save.dat"); return; }

            if (input == 'm') {
                player.switchMode();
                addLog("Switched combat mode.");
                turnTaken = false;
            }
            else if (input == 'w' || input == 's' || input == 'a' ||
input == 'd') {
```

```
                int nextX = player.getPosition().first;
                int nextY = player.getPosition().second;
                if (input == 'w') nextY--; if (input == 's') nextY++;
                if (input == 'a') nextX--; if (input == 'd') nextX++;

                if (nextX >= 0 && nextX < field.getWidth() && nextY >= 0
&& nextY < field.getHeight()) {
                    if (!field.isObstacle(nextX, nextY)) {
                        player.setPosition(nextX, nextY);
                        turnTaken = true;
                    } else addLog("Blocked by wall!");
                }
            }
            // П о к у п к а
            else if (input == 'b') {
                if (player.getScore() >= 50) {
                    std::unique_ptr<ISpell> newSpell;
                    int r = std::rand() % 3;
                    if       (r       ==       0)       newSpell       =
std::make_unique<DirectDamageSpell>();
                    else     if     (r     ==     1)     newSpell     =
std::make_unique<AreaDamageSpell>();
                    else newSpell = std::make_unique<TrapSpell>();

                    if  (player.getHand().addSpell(std::move(newSpell)))
{
                        player.addScore(-50);
                        addLog("Bought new spell!");
                        turnTaken = true;
                    } else {
                        addLog("Hand is full! (Turn not wasted)");
                    }
                } else {
                    addLog("Not enough score! (Turn not wasted)");
                }
            }
            // М а г и я
            else if (input >= '1' && input <= '3') {
                size_t idx = static_cast<size_t>(input - '1');
                if (idx < player.getHand().getSpells().size()) {
                    std::cout << "Direction (WASD) for spell: ";
                    char dir; std::cin >> dir;
                    int tx = player.getPosition().first;
                    int ty = player.getPosition().second;
                    if (dir == 'w') ty--; if (dir == 's') ty++; if (dir
== 'a') tx--; if (dir == 'd') tx++;

                    if(player.getHand().castSpell(idx,  tx,  ty,  field,
enemies)) {
                        addLog("Magic applied!");
                        turnTaken = true;
                    } else {
                        addLog("Spell   target   invalid!   (Turn   not
wasted)");
                    }
                }
            }
```

```
            else if (input == 'f') {
                int    px    =    player.getPosition().first,    py    =
player.getPosition().second;
                bool hit = false;
                for (size_t i = 0; i < enemies.size(); ++i) {
                    if  (std::abs(enemies[i].getX()  -  px)  <=  1  &&
std::abs(enemies[i].getY() - py) <= 1) {
                        enemies[i].takeDamage(player.getDamage());
                        addLog("Hit E" + std::to_string(i+1) + "!");
                        hit = true;
                        break;
                    }
                }
                if (!hit) addLog("Player swings at air...");

                turnTaken = true;
            }
        }
        catch (const GameException& e) {
            addLog(e.what());
        }

        // ВРАГИ ХОДЯТ ТОЛЬКО ЕСЛИ ИГРОК ПОТРАТИЛ
ХОД
        if (turnTaken) {
            for (auto& e : enemies) {
                e.moveTowardsPlayer(player, field);
            }
        }
    }

    void Game::update() {
        // Атака башен
        for (size_t i = 0; i < towers.size(); ++i) {
            int dmgTaken = towers[i].update(player);
            if (dmgTaken > 0) addLog("Tower T" + std::to_string(i+1) + "
shot you! -" + std::to_string(dmgTaken) + " HP");
        }

        // Ловушки
        for (auto& e : enemies) {
            if    (field.getCell(e.getX(),    e.getY()).getType()    ==
CellType::TRAP) {
                e.takeDamage(40);
                field.setCellType(e.getX(), e.getY(), CellType::EMPTY);
                addLog("Enemy hit by TRAP!");
            }
        }

        // Обработка смертей врагов
        for (auto it = enemies.begin(); it != enemies.end(); ) {
            if (it->getHp() <= 0) {
                player.addScore(100);
                addLog("Enemy defeated! +100 Score");
                it = enemies.erase(it);
            } else ++it;
        }
```

```
        // Условие победы
        if (enemies.empty()) {
            currentLevel++;
            initLevel(currentLevel);
        }

        // Условие поражения
        if (player.getHp() <= 0) {
            render();
            std::cout << "\n[ YOU DIED ] 1. Restart Level  2. Exit to
Menu" << std::endl;
            char choice; std::cin >> choice;
            if (choice == '1') {
                initLevel(currentLevel);
            } else {
                inMenu = true;
                currentLevel = 1;
            }
        }
    }
    void Game::run() {
        while (isRunning) {
            if (inMenu) {
                showMenu();
            } else {
                render();
                handleInput();
                update();
            }
        }
    }

    void Game::addLog(const std::string& msg) {
        battleLog.push_back(msg);
        if (battleLog.size() > 5) {
            battleLog.erase(battleLog.begin());
        }
    }
```

Название файла: GameExceptions.h

```
    #ifndef GAMEEXCEPTIONS_H
    #define GAMEEXCEPTIONS_H

    #include <stdexcept>
    #include <string>

    class GameException : public std::runtime_error {
    public:
        explicit     GameException(const     std::string&     msg)     :
std::runtime_error(msg) {}
    };

    // Ошибка открытия файла
    class FileException : public GameException {
    public:
```

```
    explicit FileException(const std::string& filename)
        : GameException("Error: Could not open file " + filename) {}
};

// Ошибка данных (файл поврежден)
class SaveDataException : public GameException {
public:
    SaveDataException()  :  GameException("Error:  Save  file  is
corrupted or invalid.") {}
};

#endif
```